

```

Doub beta(const Doub z, const Doub w) {
Returns the value of the beta function  $B(z, w)$ .
    return exp(gammln(z)+gammln(w)-gammln(z+w));
}

```

gamma.h

CITED REFERENCES AND FURTHER READING:

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>, Chapter 6.

Lanczos, C. 1964, "A Precision Approximation of the Gamma Function," *SIAM Journal on Numerical Analysis*, ser. B, vol. 1, pp. 86–96.[1]

6.2 Incomplete Gamma Function and Error Function

The incomplete gamma function is defined by

$$P(a, x) \equiv \frac{\gamma(a, x)}{\Gamma(a)} \equiv \frac{1}{\Gamma(a)} \int_0^x e^{-t} t^{a-1} dt \quad (a > 0) \quad (6.2.1)$$

It has the limiting values

$$P(a, 0) = 0 \quad \text{and} \quad P(a, \infty) = 1 \quad (6.2.2)$$

The incomplete gamma function $P(a, x)$ is monotonic and (for a greater than one or so) rises from “near-zero” to “near-unity” in a range of x centered on about $a - 1$, and of width about $\sqrt{a}$ (see Figure 6.2.1).

The complement of $P(a, x)$ is also confusingly called an incomplete gamma function,

$$Q(a, x) \equiv 1 - P(a, x) \equiv \frac{\Gamma(a, x)}{\Gamma(a)} \equiv \frac{1}{\Gamma(a)} \int_x^\infty e^{-t} t^{a-1} dt \quad (a > 0) \quad (6.2.3)$$

It has the limiting values

$$Q(a, 0) = 1 \quad \text{and} \quad Q(a, \infty) = 0 \quad (6.2.4)$$

The notations $P(a, x)$, $\gamma(a, x)$, and $\Gamma(a, x)$ are standard; the notation $Q(a, x)$ is specific to this book.

There is a series development for $\gamma(a, x)$ as follows:

$$\gamma(a, x) = e^{-x} x^a \sum_{n=0}^{\infty} \frac{\Gamma(a)}{\Gamma(a+1+n)} x^n \quad (6.2.5)$$

One does not actually need to compute a new $\Gamma(a+1+n)$ for each n ; one rather uses equation (6.1.3) and the previous coefficient.

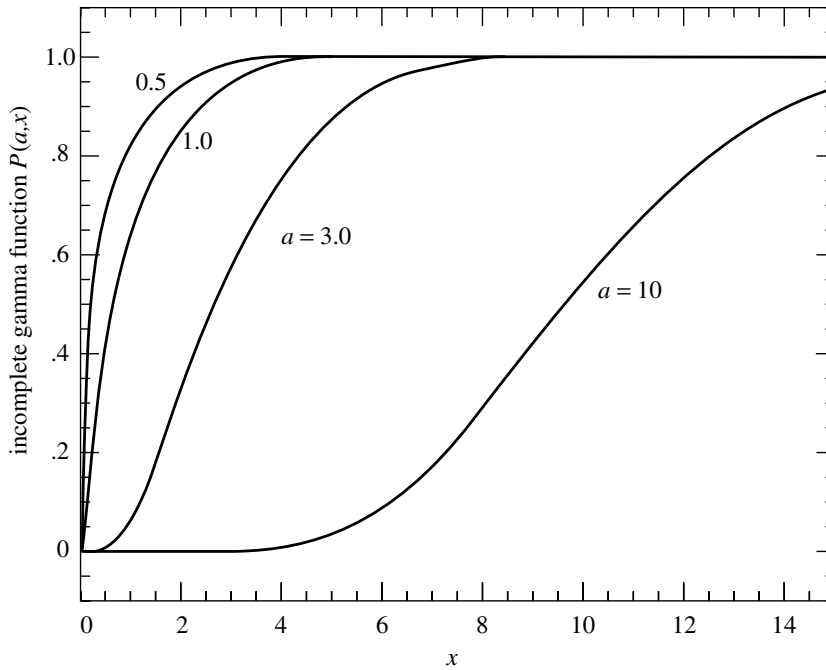


Figure 6.2.1. The incomplete gamma function $P(a, x)$ for four values of a .

A continued fraction development for $\Gamma(a, x)$ is

$$\Gamma(a, x) = e^{-x} x^a \left(\frac{1}{x+} \frac{1-a}{1+} \frac{1}{x+} \frac{2-a}{1+} \frac{2}{x+} \dots \right) \quad (x > 0) \quad (6.2.6)$$

It is computationally better to use the even part of (6.2.6), which converges twice as fast (see §5.2):

$$\Gamma(a, x) = e^{-x} x^a \left(\frac{1}{x+1-a-} \frac{1 \cdot (1-a)}{x+3-a-} \frac{2 \cdot (2-a)}{x+5-a-} \dots \right) \quad (x > 0) \quad (6.2.7)$$

It turns out that (6.2.5) converges rapidly for x less than about $a + 1$, while (6.2.6) or (6.2.7) converges rapidly for x greater than about $a + 1$. In these respective regimes each requires at most a few times $\sqrt{a}$ terms to converge, and this many only near $x = a$, where the incomplete gamma functions are varying most rapidly. For moderate values of a , less than 100, say, (6.2.5) and (6.2.7) together allow evaluation of the function for all x . An extra dividend is that we never need to compute a function value near zero by subtracting two nearly equal numbers.

Some applications require $P(a, x)$ and $Q(a, x)$ for much larger values of a , where both the series and the continued fraction are inefficient. In this regime, however, the integrand in equation (6.2.1) falls off sharply in both directions from its peak, within a few times $\sqrt{a}$. An efficient procedure is to evaluate the integral directly, with a single step of high-order Gauss-Legendre quadrature (§4.6) extending from x just far enough into the nearest tail to achieve negligible values of the integrand. Actually it is “half a step,” because we need the dense abscissas only near x , not far out on the tail where the integrand is effectively zero.

We package the various incomplete gamma parts into an object Gamma. The only persistent state is the value `gln`, which is set to $\Gamma(a)$ for the most recent call to $P(a, x)$ or $Q(a, x)$. This is useful when you need a different normalization convention, for example $\gamma(a, x)$ or $\Gamma(a, x)$ in equations (6.2.1) or (6.2.3).

```
struct Gamma : Gauleg18 {
```

[incgammabeta.h](#)

Object for incomplete gamma function. Gauleg18 provides coefficients for Gauss-Legendre quadrature.

```
    static const Int ASWITCH=100;           When to switch to quadrature method.
    static const Doub EPS;                  See end of struct for initializations.
    static const Doub FPMIN;
    Doub gln;
```

```
    Doub gammmp(const Doub a, const Doub x) {
    Returns the incomplete gamma function  $P(a, x)$ .
        if (x < 0.0 || a <= 0.0) throw("bad args in gammmp");
        if (x == 0.0) return 0.0;
        else if ((Int)a >= ASWITCH) return gammppapprox(a,x,1); Quadrature.
        else if (x < a+1.0) return gser(a,x);           Use the series representation.
        else return 1.0-gcf(a,x);           Use the continued fraction representation.
    }
```

```
    Doub gammmq(const Doub a, const Doub x) {
    Returns the incomplete gamma function  $Q(a, x) \equiv 1 - P(a, x)$ .
        if (x < 0.0 || a <= 0.0) throw("bad args in gammmq");
        if (x == 0.0) return 1.0;
        else if ((Int)a >= ASWITCH) return gammppapprox(a,x,0); Quadrature.
        else if (x < a+1.0) return 1.0-gser(a,x);       Use the series representation.
        else return gcf(a,x);           Use the continued fraction representation.
    }
```

```
    Doub gser(const Doub a, const Doub x) {
    Returns the incomplete gamma function  $P(a, x)$  evaluated by its series representation.
    Also sets  $\ln \Gamma(a)$  as gln. User should not call directly.
        Doub sum, del, ap;
        gln=gammln(a);
        ap=a;
        del=sum=1.0/a;
        for (;;) {
            ++ap;
            del *= x/ap;
            sum += del;
            if (fabs(del) < fabs(sum)*EPS) {
                return sum*exp(-x+a*log(x)-gln);
            }
        }
    }
```

```
    Doub gcf(const Doub a, const Doub x) {
    Returns the incomplete gamma function  $Q(a, x)$  evaluated by its continued fraction representation. Also sets  $\ln \Gamma(a)$  as gln. User should not call directly.
        Int i;
        Doub an,b,c,d,del,h;
        gln=gammln(a);
        b=x+1.0-a;
        c=1.0/FPMIN;
        d=1.0/b;
        h=d;
        for (i=1;;i++) {
            an = -i*(i-a);
            b += 2.0;
            d=an*d+b;
            Set up for evaluating continued fraction
            by modified Lentz's method (§5.2)
            with  $b_0 = 0$ .

            Iterate to convergence.
        }
```

```

        if (fabs(d) < FPMIN) d=FPMIN;
        c=b+an/c;
        if (fabs(c) < FPMIN) c=FPMIN;
        d=1.0/d;
        del=d*c;
        h *= del;
        if (fabs(del-1.0) <= EPS) break;
    }
    return exp(-x+a*log(x)-gln)*h;          Put factors in front.
}

Doub gammapprox(Doub a, Doub x, Int psig) {
    Incomplete gamma by quadrature. Returns  $P(a, x)$  or  $Q(a, x)$ , when psig is 1 or 0,
    respectively. User should not call directly.
    Int j;
    Doub xu,t,sum,ans;
    Doub a1 = a-1.0, lna1 = log(a1), sqarta1 = sqrt(a1);
    gln = gammln(a);
    Set how far to integrate into the tail:
    if (x > a1) xu = MAX(a1 + 11.5*sqarta1, x + 6.0*sqarta1);
    else xu = MAX(0., MIN(a1 - 7.5*sqarta1, x - 5.0*sqarta1));
    sum = 0;
    for (j=0;j<ngau;j++) {                    Gauss-Legendre.
        t = x + (xu-x)*y[j];
        sum += w[j]*exp(-(t-a1)+a1*(log(t)-lna1));
    }
    ans = sum*(xu-x)*exp(a1*(lna1-1.0)-gln);
    return (psig?(ans>0.0? 1.0-ans:-ans):(ans>=0.0? ans:1.0+ans));
}

Doub invgamm(Doub p, Doub a);
Inverse function on  $x$  of  $P(a, x)$ . See §6.2.1.
};
const Doub Gamma::EPS = numeric_limits<Doub>::epsilon();
const Doub Gamma::FPMIN = numeric_limits<Doub>::min()/EPS;

```

Remember that since Gamma is an object, you have to declare an instance of it before you can use its member functions. We habitually write

```
Gamma gam;
```

as a global declaration, and then call `gam.gamm` or `gam.gammq` as needed. The structure `GauLeg18` just contains the abscissas and weights for the Gauss-Legendre quadrature.

ncgammabeta.h

```

struct GauLeg18 {
    Abscissas and weights for Gauss-Legendre quadrature.
    static const Int ngau = 18;
    static const Doub y[18];
    static const Doub w[18];
};
const Doub GauLeg18::y[18] = {0.0021695375159141994,
0.011413521097787704,0.027972308950302116,0.051727015600492421,
0.082502225484340941, 0.12007019910960293,0.16415283300752470,
0.21442376986779355, 0.27051082840644336, 0.33199876341447887,
0.39843234186401943, 0.46931971407375483, 0.54413605556657973,
0.62232745288031077, 0.70331500465597174, 0.78649910768313447,
0.87126389619061517, 0.95698180152629142};
const Doub GauLeg18::w[18] = {0.0055657196642445571,
0.012915947284065419,0.020181515297735382,0.027298621498568734,
0.034213810770299537,0.040875750923643261,0.047235083490265582,
0.053244713977759692,0.058860144245324798,0.064039797355015485,

```

```
0.068745323835736408,0.072941885005653087,0.076598410645870640,
0.079687828912071670,0.082187266704339706,0.084078218979661945,
0.085346685739338721,0.085983275670394821};
```

6.2.1 Inverse Incomplete Gamma Function

In many statistical applications one needs the inverse of the incomplete gamma function, that is, the value x such that $P(a, x) = p$, for a given value $0 \leq p \leq 1$. Newton's method works well if we can devise a good-enough initial guess. In fact, this is a good place to use Halley's method (see §9.4), since the second derivative (that is, the first derivative of the integrand) is easy to compute.

For $a > 1$, we use an initial guess that derives from §26.2.22 and §26.4.17 in reference [1]. For $a \leq 1$, we first roughly approximate $P_a \equiv P(a, 1)$:

$$P_a \equiv P(a, 1) \approx 0.253a + 0.12a^2, \quad 0 \leq a \leq 1 \quad (6.2.8)$$

and then solve for x in one or the other of the (rough) approximations:

$$P(a, x) \approx \begin{cases} P_a x^a, & x < 1 \\ P_a + (1 - P_a)(1 - e^{1-x}), & x \geq 1 \end{cases} \quad (6.2.9)$$

An implementation is

```
Doub Gamma::invgammp(Doub p, Doub a) { incgammabeta.h
Returns x such that  $P(a, x) = p$  for an argument  $p$  between 0 and 1.
    Int j;
    Doub x, err, t, u, pp, lna1, afac, a1=a-1;
    const Doub EPS=1.e-8; Accuracy is the square of EPS.
    gln=gammln(a);
    if (a <= 0.) throw("a must be pos in invgammp");
    if (p >= 1.) return MAX(100., a + 100.*sqrt(a));
    if (p <= 0.) return 0.0;
    if (a > 1.) { Initial guess based on reference [1].
        lna1=log(a1);
        afac = exp(a1*(lna1-1.)-gln);
        pp = (p < 0.5)? p : 1. - p;
        t = sqrt(-2.*log(pp));
        x = (2.30753+t*0.27061)/(1.+t*(0.99229+t*0.04481)) - t;
        if (p < 0.5) x = -x;
        x = MAX(1.e-3, a*pow(1.-1./(9.*a)-x/(3.*sqrt(a)), 3));
    } else { Initial guess based on equations (6.2.8)
        t = 1.0 - a*(0.253+a*0.12); and (6.2.9).
        if (p < t) x = pow(p/t, 1./a);
        else x = 1.-log(1.-(p-t)/(1.-t));
    }
    for (j=0; j<12; j++) {
        if (x <= 0.0) return 0.0; x too small to compute accurately.
        err = gammp(a, x) - p;
        if (a > 1.) t = afac*exp(-(x-a1)+a1*(log(x)-lna1));
        else t = exp(-x+a1*log(x)-gln);
        u = err/t;
        x -= (t = u/(1.-0.5*MIN(1., u*((a-1.)/x - 1))))); Halley's method.
        if (x <= 0.) x = 0.5*(x + t); Halve old value if x tries to go negative.
        if (fabs(t) < EPS*x ) break;
    }
    return x;
}
```

6.2.2 Error Function

The error function and complementary error function are special cases of the incomplete gamma function and are obtained moderately efficiently by the above procedures. Their definitions are

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \quad (6.2.10)$$

and

$$\operatorname{erfc}(x) \equiv 1 - \operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-t^2} dt \quad (6.2.11)$$

The functions have the following limiting values and symmetries:

$$\operatorname{erf}(0) = 0 \quad \operatorname{erf}(\infty) = 1 \quad \operatorname{erf}(-x) = -\operatorname{erf}(x) \quad (6.2.12)$$

$$\operatorname{erfc}(0) = 1 \quad \operatorname{erfc}(\infty) = 0 \quad \operatorname{erfc}(-x) = 2 - \operatorname{erfc}(x) \quad (6.2.13)$$

They are related to the incomplete gamma functions by

$$\operatorname{erf}(x) = P\left(\frac{1}{2}, x^2\right) \quad (x \geq 0) \quad (6.2.14)$$

and

$$\operatorname{erfc}(x) = Q\left(\frac{1}{2}, x^2\right) \quad (x \geq 0) \quad (6.2.15)$$

A faster calculation takes advantage of an approximation of the form

$$\operatorname{erfc}(z) \approx t \exp[-z^2 + \mathcal{P}(t)], \quad z > 0 \quad (6.2.16)$$

where

$$t \equiv \frac{2}{2+z} \quad (6.2.17)$$

and $\mathcal{P}(t)$ is a polynomial for $0 \leq t \leq 1$ that can be found by Chebyshev methods (§5.8). As with Gamma, implementation is by an object that also includes the inverse function, here an inverse for both erf and erfc. Halley's method is again used for the inverses (as suggested by P.J. Acklam).

```

erf.h  struct Erf {
      Object for error function and related functions.
      static const Int ncof=28;
      static const Doub cof[28];                                Initialization at end of struct.

      inline Doub erf(Doub x) {
      Return erf(x) for any x.
          if (x >= 0.) return 1.0 - erfccheb(x);
          else return erfccheb(-x) - 1.0;
      }

      inline Doub erfc(Doub x) {
      Return erfc(x) for any x.
          if (x >= 0.) return erfccheb(x);
          else return 2.0 - erfccheb(-x);
      }

```

```
Doub erfccheb(Doub z){
```

Evaluate equation (6.2.16) using stored Chebyshev coefficients. User should not call directly.

```
    Int j;
    Doub t,ty,tmp,d=0.,dd=0.;
    if (z < 0.) throw("erfccheb requires nonnegative argument");
    t = 2./(2.+z);
    ty = 4.*t - 2.;
    for (j=ncof-1;j>0;j--) {
        tmp = d;
        d = ty*d - dd + cof[j];
        dd = tmp;
    }
    return t*exp(-z*z + 0.5*(cof[0] + ty*d) - dd);
}
```

```
Doub inverfc(Doub p) {
```

Inverse of complementary error function. Returns x such that $\text{erfc}(x) = p$ for argument p between 0 and 2.

```
    Doub x,err,t,pp;
    if (p >= 2.0) return -100.;           Return arbitrary large pos or neg value.
    if (p <= 0.0) return 100.;
    pp = (p < 1.0)? p : 2. - p;
    t = sqrt(-2.*log(pp/2.));              Initial guess:
    x = -0.70711*((2.30753+t*0.27061)/(1.+t*(0.99229+t*0.04481)) - t);
    for (Int j=0;j<2;j++) {
        err = erfc(x) - pp;
        x += err/(1.12837916709551257*exp(-SQR(x))-x*err); Halley.
    }
    return (p < 1.0? x : -x);
}
```

```
inline Doub inverf(Doub p) {return inverfc(1.-p);}
```

Inverse of the error function. Returns x such that $\text{erf}(x) = p$ for argument p between -1 and 1 .

```
};
```

```
const Doub Erf::cof[28] = {-1.3026537197817094, 6.4196979235649026e-1,
    1.9476473204185836e-2,-9.561514786808631e-3,-9.46595344482036e-4,
    3.66839497852761e-4,4.2523324806907e-5,-2.0278578112534e-5,
    -1.624290004647e-6,1.303655835580e-6,1.5626441722e-8,-8.5238095915e-8,
    6.529054439e-9,5.059343495e-9,-9.91364156e-10,-2.27365122e-10,
    9.6467911e-11, 2.394038e-12,-6.886027e-12,8.94487e-13, 3.13092e-13,
    -1.12708e-13,3.81e-16,7.106e-15,-1.523e-15,-9.4e-17,1.21e-16,-2.8e-17};
```

A lower-order Chebyshev approximation produces a very concise routine, though with only about single precision accuracy:

```
Doub erfcc(const Doub x)
```

Returns the complementary error function $\text{erfc}(x)$ with fractional error everywhere less than 1.2×10^{-7} .

```
{
    Doub t,z=fabs(x),ans;
    t=2./(2.+z);
    ans=t*exp(-z*z-1.26551223+t*(1.00002368+t*(0.37409196+t*(0.09678418+
        t*(-0.18628806+t*(0.27886807+t*(-1.13520398+t*(1.48851587+
        t*(-0.82215223+t*0.17087277)))))))));
    return (x >= 0.0 ? ans : 2.0-ans);
}
```

erf.h